Architecture de Tessera Santé

Document de référence pour le développement. Repris de la fiche d'architecture établie avec le porteur du 4 au 7 novembre 2026 ; à tenir à jour à chaque décision. Les fiches projet et technique en sont les versions destinées à la direction des soins et au service informatique.

1. Objet et principes

Le code de **Suivi-mesures-isocont** (dépôt `rodtrip2/Suivi-mesures-isocont`) devient le premier module de la plateforme Tessera Santé : un **socle commun** (connexion, comptes, services, droits, chiffrement, stockage, sauvegardes, journal) et des **modules** métier qui s'y branchent. Une seule connexion ouvre les modules auxquels le compte a droit.

- **Aucun recul sur les garanties d'Isocont** : application statique, aucune connexion sortante, données chiffrées, journal en ajout seul, saisie simultanée sans perte.
- **Un droit = une clé** : un compte ne peut déchiffrer que les modules auxquels il a accès.
- **Frontière nette** : le socle ne connaît aucune règle métier ; un module ne touche ni au coffre, ni aux clés, ni aux comptes.
- **Modules par service** : l'établissement choisit quels modules sont ouverts dans quel service.
- **Échanges déclarés** : un module ne lit jamais le journal d'un autre ; il demande au socle une information que l'autre module a déclarée comme fournie.
- **Isocont fonctionne à chaque étape** du développement : chaque étape est une version publiable, sans perte de données.
- **Mêmes conventions** qu'Isocont : sans dépendance ni étape de construction, logique testable sous Node.
- **Ouverture** : la fiche patient et le socle ne supposent pas la psychiatrie.

2. État des lieux du code d'Isocont

Une dizaine de fichiers sont déjà indépendants du métier ; le mode formation applique déjà le principe d'un jeu de données à clé propre. Trois fichiers centraux (`model.js`, `session.js`, `ui.js`) mêlent socle et métier et doivent être découpés.

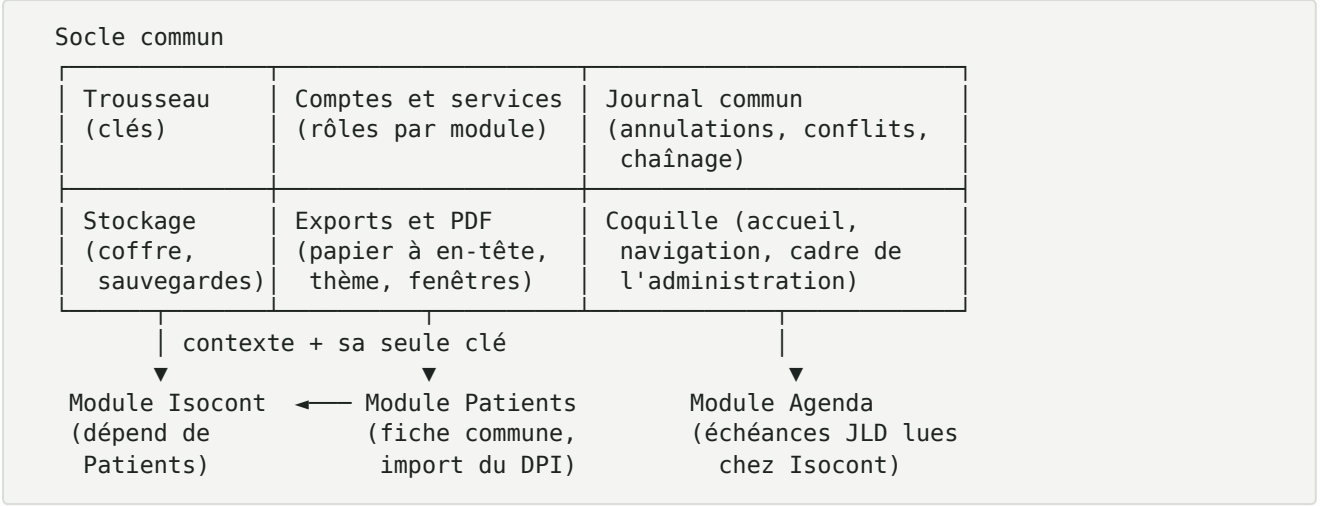
Trois constats :

1. **Une seule clé de données pour tout** : `coffre.json` enveloppe une clé unique (mot de passe, YubiKey, code de secours) qui chiffre les comptes et toutes les saisies.
2. **Services, rôles et accès vivent dans le journal d'Isocont** (saisies `CONFIG` : `config.services`, `config.roles`, `M.ACCES`) : ils passent au socle.
3. **Le mode formation est le bon patron** : sous-dossier `formation/`, clé propre enveloppée dans `coffre.formation`, bascule `dossierActif` / `cleActive` dans `Session`. Un module reprend cette mécanique.

Fichier d'Isocont	Lignes (nov. 2026)	Destination	Ce qui change
`crypto.js`	194	Socle, tel quel	Ajouter les clés personnelles (ECDH)

Fichier d'Isocont	Lignes (nov. 2026)	Destination	Ce qui change
`comptes.js`	141	Socle, tel quel	—
`xlsx.js`, `pdf.js`	149, 277	Socle, tel quel	Bibliothèques partagées
`theme.js`, `ui-version.js`, `ui-hors-reseau.js`, `ui-verrou.js`, `sw.js`	13 à 117	Socle, tel quel	Libellés à généraliser
`store.js`	699	Socle, à adapter	Arborescence par module, noms réservés
`session.js`	684	À découper	Coffre, connexion, intégrité, miroir, exports, préférences → socle ; `enregistrer`, `recalculer`, `peut` appellent le module
`model.js`	2 412	À découper	Rôles, accès, services, réduction du journal, annulations, conflits, chaînage → socle (environ un cinquième) ; le reste → Isocont
`ui.js`	2 823	À découper	Accueil, connexion, en-tête, navigation, cadre de l'administration → socle ; vues métier → Isocont
`ui-comptes.js`, `ui-roles.js`, `ui-formation.js`	67 à 150	Socle	`ui-roles.js` lit les accès déclarés par chaque module au lieu de `M.ACCES`
`ui-sauvegarde.js`, `ui-miroir.js`, `ui-reglages.js`, `ui-pdf-etab.js`, `ui-apropos.js`	54 à 355	Socle	Appels au modèle remplacés par le contrat de module
`rules.js`, `import-v13.js`, `import-registre.js`, `demo.js`, `ui-patients.js`, `ui-decompte.js`, `ui-stats.js`, `ui-registre-cada.js`, `ui-alertes.js`, `ui-cas.js`, `ui-parcours.js`, `ui-qualite.js`, `ui-methode.js`, `ui-import.js`	—	Module Isocont (fiches patient → module Patients)	Pas de changement de fond

3. Architecture cible



Ce que le socle fournit au module (le contexte)

- L'agent connecté, ses services autorisés et `peut(acces)` pour ce module.
- `enregistrer(type, data, base)` : écrit une saisie chiffrée avec la clé du module, chaînée et horodatée par le socle.
- La réduction générique du journal : tri, annulations et rétablissements, chaînage, détection des conflits (le module dit seulement quel champ une saisie modifie).
- `exporter`, le générateur PDF avec papier à en-tête, l'aide `h()`, les fenêtres, le thème.

Ce que le module déclare (son contrat)

Élément	Contenu	Exemple pour Isocont
Identité	identifiant, libellé, version, icône	`isocont`, « Isolement et contention »
Accès	accès cochables pour les rôles	les 16 accès actuels de `M.ACCES`
Saisies	types, champ logique pour les conflits, fonction d'application	`MESURE_DEBUT`, `JLD`, `PAUSE`…
Réglages	valeurs par défaut, validation, groupes exportables	`CONFIG_DEFAUT`, `M.GROUPES_REGLAGES`
Vues	onglets et sous-onglets d'administration, chacun lié à un accès	Tableau de bord, Mesures, JLD, Registre…
Migrations	étapes numérotées et idempotentes	reprise des réglages actuels
Formation	génération d'un jeu fictif (facultatif)	`demo.js`
Fournitures	nom, version, champs renvoyés, accès requis	`isocont.echeancesJLD` v1
Besoins	fourniture attendue, facultative ou non	`patients` (obligatoire)

Les saisies propres au socle (`COMPTE`, réglages de l'établissement et des services, activation des modules, `INTEGRITE`) restent dans le journal du socle. Tant qu'un seul module est actif, l'écran d'accueil des modules est sauté.

Administration unique : un onglet général pour les éléments communs (comptes, services, activation des modules, sécurité, sauvegardes) et un onglet par module, fourni par le module.

4. Comptes, droits et clés

Clé	Qui la détient	Ce qu'elle ouvre
Clé personnelle (paire ECDH P-256, Web Crypto)	l'agent : partie privée emballée par son mot de passe ou sa YubiKey ; partie publique en clair dans le coffre	son trousseau
Clé de plateforme (AES-GCM 256)	tous les comptes actifs	comptes, services, rôles affectés, journal du socle (aucune donnée de patient)
Clé d'un module (AES-GCM 256, par générations)	les agents qui y ont droit, et l'administrateur	les saisies et réglages de ce module
Code de secours (**un seul pour la plateforme**)	l'établissement, sous scellé	la clé de plateforme et toutes les clés de module

- **Remise d'une clé** : l'administrateur chiffre la clé du module avec la partie publique de l'agent (ECDH + HKDF-SHA-256), sans connaître son mot de passe, sur le poste, sans serveur.
- **Rôles** : chaque module définit ses rôles à partir de ses accès ; le socle affecte au compte ses services (communs à tous les modules) et, module par module, ses rôles. Sans rôle dans un module, pas de clé et pas d'affichage.
- **Retrait d'un droit** : suppression de l'enveloppe, puis **nouvelle génération** de la clé du module pour les saisies à venir. Toutes les générations sont remises aux agents qui gardent l'accès et à tout nouvel agent : ils lisent tout l'historique ; seul l'agent retiré ne reçoit pas la nouvelle.
- **Isocont à la migration** : sa clé de module peut être l'actuelle clé de données (pas de rechiffrement). L'outil n'étant pas déployé, la mise en production part toutefois d'un dossier vide.
- **Mode formation** : propre à chaque module, clé du jeu fictif enveloppée par la clé du module.
- **Décisions du porteur pour l'étape 4** (7 novembre 2026, plan relu : *Tessera Santé — Plan de l'étape 4 : le trousseau de clés*) : 1. **Générations tirées d'une clé maîtresse** par clé de droit (HKDF-SHA-256), détenue par l'administrateur et par une **clé de secours** (paire ECDH dont la partie privée est emballée par le code de secours) : la copie sous scellé reste valable après chaque nouvelle génération ; seul le retrait d'un administrateur impose une nouvelle clé maîtresse. 2. **Nouvelle génération automatique** au retrait d'un rôle ou à la désactivation d'un compte, annoncée dans la fenêtre de confirmation. 3. **Aucun rechiffrement** à la migration : l'ancienne clé de données devient la génération 1 des clés de plateforme et d'Isocont ; seules les saisies postérieures sont cloisonnées. 4. Migration des dossiers d'essai : **mots de passe provisoires** pour tous les comptes (sans objet en production, qui part d'un dossier vide). 5. **YubiKey de secours** non nominative **conservée**, rangée avec le code de secours (elle ouvre la clé de secours). 6. **Nouvelle paire de clés créée par l'agent** à son premier changement de mot de passe : l'administrateur ne connaît jamais la clé privée définitive. 7. Clés de module : agents ayant un rôle dans le module et **administrateur seul** en plus.
- **Fichiers chiffrés** : en-tête « SIC2 » portant le numéro de génération (lié au contenu) ; les fichiers « SIC1 » sont lus comme génération 1.
- **Coffre version 2** (0.4.2, `trousseau.js`) : `agents` (parties publiques, compte et clé de secours), `emballages` (partie privée d'un agent ou de la clé de secours, par mot de passe, YubiKey ou code de secours), `cles` (génération courante de chaque clé de droit), `enveloppes` (`cle`, `g` = génération ou `m` pour la clé maîtresse, `pour`), `comptes` chiffrés par la clé de plateforme. Un

dossier neuf est créé en version 2 ; un coffre version 1 s'ouvre et s'utilise comme avant (fichiers « SIC1 ») jusqu'à sa migration (0.4.3).

- **Migration d'un coffre version 1** (0.4.3, *Comptes et sécurité → Protection des données*) : par un compte administrateur, qui redonne son mot de passe (il le garde) et, s'il le connaît, le code de secours actuel (repris ; sinon nouveau code). L'ancienne clé de données devient la génération 1 « d'origine » des clés de plateforme et d'Isocont (enveloppée aussi pour la clé de secours, car elle ne se déduit pas d'une clé maîtresse) ; génération 2 distincte pour les saisies à venir ; clé personnelle et mot de passe provisoire pour chaque autre compte actif (fiches de remise) ; clés de sécurité à réenregistrer, copie sous scellé à refaire ; trace `MIGRATION_TROUSSEAU`. Un poste resté connecté se verrouille à sa synchronisation suivante. **Retour à la clé unique** : tant que l'ancien coffre figure dans l'historique, l'administrateur peut le remettre en place (les saisies postérieures à la migration deviennent illisibles).
- **Remise selon les droits** (0.4.4) : clé de plateforme à tout compte actif ; clé d'Isocont à un compte qui a un rôle d'Isocont et au moins un service où le module est activé (l'encadrement et l'administrateur suivent l'activation ; l'administrateur a toujours toutes les clés et les clés maîtresses). Sans la clé, le journal du module n'est pas lu et l'agent voit un message. Les droits sont appliqués aux clés à chaque modification d'un compte et après chaque réglage, annulation ou import (`Session.appliquerDroits`, sans effet si rien ne change) : un détenteur qui perd une clé → nouvelle génération remise aux seuls détenteurs restants (trace `GENERATION`) ; un administrateur qui perd ce rôle → nouvelle clé maîtresse (`cles.<nom>.base` : première génération qu'elle produit ; les générations antérieures sont remises explicitement, au secours compris).
- **Premier changement de mot de passe** : l'agent crée sa propre clé personnelle et se remet ses clés ; l'administrateur ne l'a jamais connue.
- **Postes ouverts** : à la relecture du coffre (au plus toutes les minutes), le trousseau est rouvert ; accès retiré ou clé personnelle renouvelée → verrouillage avec message. Délai assumé : un poste peut écrire jusqu'à une minute avec l'ancienne génération.
- **Trousseaux incomplets** : listés dans *Comptes et sécurité* (« Clés à réémettre »), réémission par l'administrateur.
- Garde-fou : une clé dont les octets sont tous nuls est refusée (aucun chiffrement avec une clé effacée par erreur).
- **En mémoire pendant la session** : les clés de droit ouvertes et la partie privée de l'agent (nécessaire pour réemballer sa clé : changement de mot de passe, ajout d'une YubiKey), effacées au verrouillage ; les clés AES importées ne sont pas exportables.
- **Relecture indépendante** (0.4.5) : corrections appliquées. Comptes expirés traités comme désactivés pour les clés ; tout changement de son mot de passe crée une **nouvelle clé personnelle** (les clés de sécurité de l'agent sont retirées, à réenregistrer) ; retrait de sa propre clé de sécurité = mot de passe redemandé et nouvelle clé personnelle (une clé perdue n'ouvre plus rien) ; la clé de sécurité d'un autre agent se retire en réinitialisant son mot de passe ; connexion par clé de sécurité refusée pour un compte expiré ou désactivé ; avant chaque écriture du coffre, le trousseau du poste est confronté au coffre relu (clés changées → réouverture, clé personnelle retirée ou remplacée → verrouillage) ; la génération courante ne recule jamais (la plus haute reçue fait foi) ; une enveloppe illisible n'empêche plus d'ouvrir les autres ; la copie sous scellé est signalée « à refaire » quand une clé maîtresse change.
- **Attestation du coffre** (0.5.0, `attestation.js` ; décision du porteur du 7 novembre 2026 : un agent qui garde un accès en écriture au partage et la clé de plateforme fait partie de la menace couverte). `coffre.attestation` = { `publique` (ECDSA P-256), `temoin` (ECDH), `privees` (chiffrées par une clé tirée de la clé maîtresse de plateforme), `etat` (texte canonique signé), `signature` }. L'état signé porte : date de création du coffre, compteur `n`, auteur, clé de secours, témoin, parties publiques des agents, `cles` (génération courante, base, générations perdues),

empreinte SHA-256 de chaque génération et de chaque clé maîtresse, code d'authentification des droits des comptes (rôles, services, activité, fin de validité ; clé tirée de la clé de signature : aucun rôle en clair). **Ancre** : empreinte de la clé publique d'attestation, emballée avec la partie privée de chaque agent et de la clé de secours (« ANC1 »). Tout agent : signature et ancre vérifiées à l'ouverture, seules les générations attestées acceptées, génération d'écriture = celle de l'état signé, connexion refusée sur une anomalie bloquante. Administrateur : vérifie en plus comptes, parties publiques (preuves de continuité acceptées) et clé de secours avant chaque modification du coffre, puis signe ; il se connecte malgré une anomalie pour réparer, sans aucune remise de clé avant. **Preuve de continuité** : un agent qui crée sa clé personnelle joint un HMAC tiré d'un échange ECDH entre son ancienne partie privée et le témoin. **Version antérieure** : compteur retenu par chaque poste (`localStorage`) et lu dans l'historique du coffre. Coffre d'avant 0.5.0 : attestation établie à la connexion d'un administrateur, emballages ancrés à la connexion suivante de chaque agent.

- **Réparations** (*Comptes et sécurité → Intégrité du coffre*, bandeau rouge pour l'administrateur) : *Renouveler les clés* (coffre restauré ou version antérieure ; décision du porteur : renouvellement obligatoire après toute restauration, les agents ne se connectent pas avant) ; *Nouvelle clé personnelle* d'un compte dont la partie publique a été remplacée ; *Nouveau code de secours* ; *Remettre la dernière version attestée* (version de l'historique au compteur le plus élevé dont signature, comptes et parties publiques sont intacts, la clé personnelle de l'administrateur qui répare est gardée, puis renouvellement). Renouvellement : nouvelles clés maîtresses, génération au-dessus de toute génération déjà employée (historique), générations dont l'empreinte n'est pas attestée déclarées perdues (`cles.<nom>.perdues`). Traces `ATTESTATION`, `RENOUVELLEMENT`, `REMISE_ATTESTEE`.
- **Compromission** (décision du porteur) : *Tout renouveler* : clés maîtresses et générations, clés d'attestation (nouvelle ancre), code de secours, clé personnelle de chaque compte actif ; l'administrateur garde son mot de passe, les autres reçoivent un mot de passe provisoire ; clés de sécurité retirées ; trace `COMPROMISSION`.
- **Limites assumées** : un détenteur de la clé de plateforme peut forger une saisie du journal du socle (réglage commun, dont l'activation des modules : au plus la clé d'un module où il a déjà un rôle) ; les enveloppes « m » révèlent quels comptes sont administrateurs ; les octets des clés reçues restent en mémoire pendant la session.

5. Modules par service

Un agent voit un module dans un service si : le module est **activé pour ce service**, l'agent est **affecté à ce service**, et il a **un rôle dans ce module**. Activation réglée par l'administrateur, tracée au journal du socle.

- **Clés** : l'agent reçoit la clé d'un module dès qu'un de ses services l'active.
- **Cloisonnement entre services d'un même module : par l'interface** (décision du porteur, pour simplifier les transferts de patients). Une clé par module et par service a été écartée.
- **Rôle selon le service** : prévu dans le modèle de données dès le départ (rôle affecté par module et par service) ; écran de réglage plus tard.
- **Désactivation d'un module pour un service** : données conservées, plus de saisie, consultation par le cadre et l'administrateur.
- **Réglage** (0.4.0) : `modulesServices` au journal du socle, `{ <module>: { tous: false, services: [codes activés] } }` ; sans réglage, module activé pour tous les services, y compris ceux créés ensuite ; dès qu'un module est désactivé quelque part, un service créé ensuite est à activer (choix prudent, visible dans le panneau).

- **Consultation d'un service désactivé** : l'administrateur et les rôles d'encadrement déclarés par le module (Isocont : cadre de santé). Une saisie est refusée dès qu'elle agit sur un service désactivé : service indiqué, service qui suit la mesure, service de la fiche patient (`saisies.cibles` du contrat).

6. Échanges entre modules

Mode	Fonctionnement	État
Lecture à la volée	le socle demande au module fournisseur de calculer, sur le poste, avec sa clé déjà dans le trousseau de l'agent ; rien n'est écrit	**retenu**
Publication	le fournisseur réécrit un extrait minimal chiffré par une clé d'échange du service (`echanges/<module>/<service>.enc`)	prévu dans le contrat, **désactivé par défaut** ; activation après avis du délégué à la protection des données

- Exemple : `isocont.echeancesJLD` v1 (date et heure, nature : information, saisine, décision ; type de mesure ; service ; référence de la mesure ; statut), consommé par l'Agenda, facultatif pour lui. Les échéances restent calculées par le moteur légal (`M.suiviJLD`, `rules.echeances`) depuis le début de la mesure.
- **Identité du patient dans l'agenda** : au choix de l'établissement, aucune, initiales, IPP ou nom complet (le plus exposant, à réserver aux services qui en ont besoin). Par défaut : la référence de la mesure ; un clic ouvre la mesure pour qui y a accès.
- **Lecture seule** : un module n'écrit jamais dans un autre.
- **Mise à jour** : le socle signale « module X a changé » (`Session.sur`), le consommateur se rafraîchit.
- **Garde-fous** : interfaces numérotées, pas de dépendance circulaire, tests sous Node avec un faux fournisseur.

7. Module Patients

Module de référence, partagé par tous les modules qui concernent des patients, avec ou sans mesure. La liste des patients existe dans le DPI ; la fiche commune sert à **relier les modules entre eux**.

Ce qui change de place

Élément	Aujourd'hui dans Isocont	Demain
Identité (IPP, nom, prénom, date de naissance)	saisie `PATIENT`	Patients
Protection juridique, patient détenu	saisie `PATIENT`	Patients
Mode d'hospitalisation et historique daté	saisie `MODE_HOSPI`	Patients
Fusion de fiches, doublons	`FUSION`, `M.doublonsPossibles`	Patients ; les modules rattachent leurs données à la fiche conservée

Élément	Aujourd'hui dans Isocont	Demain
Droit d'accès d'un patient	`M.donneesPatient`, Isocont seul	le socle réunit l'export de tous les modules
Durée de conservation	`M.auDelaConservation`, d'après les mesures	Patients, d'après la dernière activité dans tous les modules
Cumuls, âge au début, mode retenu au registre, alertes	moteur Isocont	restent dans Isocont, qui lit la fiche

Champs de la fiche (décision du 5 novembre 2026)

Champ	Source
IPP, nom, prénom, noms de naissance et d'usage, date de naissance, sexe	import du DPI ou saisie
Séjour : entrée, sortie, unité ; chambre et lit	import du DPI ou saisie
Médecin de 1re ligne, médecin de 2e ligne	saisie dans Tessera (distincts du médecin responsable du DPI) ; importables plus tard si le DPI les porte
Référents soignants (plusieurs)	saisie dans Tessera, parmi les comptes du service, nom libre possible
Protection juridique, patient détenu, mode d'hospitalisation	saisie dans Tessera

Règles

- **Dépendance forte** : Isocont et l'Agenda ne fonctionnent pas sans Patients ; le socle charge Patients d'abord ; un agent qui a accès à un module lié aux patients reçoit la clé de Patients en lecture.
- **Lecture et écriture distinctes** : créer ou corriger une fiche est un accès du module Patients, paramétrable (IDE, secrétaire et cadre par défaut).
- **Commun ou propre** : Patients ne garde que ce qui sert à plusieurs modules ; un module peut déclarer un champ supplémentaire, activé par l'administrateur. Aucune donnée clinique.
- **Libellés réglables par service** (« Psychiatre de 1re ligne » en psychiatrie, « Médecin de 1re ligne » ailleurs) ; liste des médecins du service reprise de `config.psychiatres` et passée au socle. Champs propres à la psychiatrie (mode légal, détenu) activés par service.
- **Historique daté, jamais écrasé** : un changement de mode, de protection, de médecin ou de référent est une saisie datée ; les calculs d'Isocont lisent la valeur en vigueur à la date de la mesure.
- **Fusion** : faite dans Patients, signalée aux modules, qui recalculent ; aperçu de l'impact sur tous les modules.

Sous-étapes (étape 5, chacune publiable) :

1. **Journal et clé du module Patients (0.6.0).** `module-patients.js` : module de référence (`reference: true` : ni onglet ni droits propres à ce stade), saisies `PATIENT`, `MODE_HOSPI`, `FUSION`, fourniture `patients.fiches` v1 ; Isocont la déclare comme besoin obligatoire. Registre : ordre de chargement (fournisseurs d'abord), module de chaque type de saisie, modules dépendants ; un type n'appartient qu'à un module. Session : un journal par module dont l'agent détient la clé

(`modules\patients\` avant `modules\isocont\`), saisie rangée dans le journal du module qui déclare son type ; Isocont reçoit les saisies des deux journaux et reconstitue son état comme avant (lecture à la volée par la session). Clé de Patients remise à qui a droit à un module qui en dépend (et à l'administrateur) ; nouvelle génération au retrait. Coffre créé avant le module : clé créée à la connexion d'un administrateur (trace `CLE_MODULE`), fiches écrites dans le journal d'Isocont en attendant. Reste dans le journal d'Isocont : les lots (imports, jeu de démonstration), jusqu'à la sous-étape 2.

2. **Fiche reconstituée par Patients (0.6.1).** Fourniture `patients.fiches` v1 calculée par Patients (`calculer`, lue par `registre.fournir`) : fiches en vigueur, fusions, fiches fusionnées, modes datés, **historique daté** de la protection, du statut de détenu et du service (`historique[champ]`, `valeurAu` ; Isocont lit encore la valeur actuelle, comme avant). Isocont (`M.reduire(evts, contexte)`) part de ces fiches et ne traite plus `PATIENT`, `MODE_HOSPI`, `FUSION` ; les mesures d'une fiche fusionnée sont rattachées à la fiche conservée dès leur lecture (fusion « signalée » par le recalcul). **Lots** : un lot touchant plusieurs journaux est rangé en un lot par journal (socle, Patients, Isocont), liés par `lien` ; annuler l'un annule les autres, les annulations produites citent l'annulation d'origine (`origine`) et se rétablissent ensemble. Droit d'accès : changements datés de la fiche ajoutés à l'export.
3. **Fiche commune complète.** *Fait (0.6.2)* : droits de la fiche (accès « Fiches patient : créer et corriger » = `PATIENT` ; mode d'hospitalisation rattaché à l'accès des mesures ; AS sans fiche, secrétaire avec, par défaut ; la fiche suit un transfert sans ce droit) ; onglet *Patients* déclaré par le module Patients et affiché en tête (`registre.onglets()`). *Fait (0.6.3)* : champs `nomNaissance`, `sexe` (F, M, I), `medecin1`, `medecin2`, `referents` ([{ id du compte ou null, nom }]) de la saisie `PATIENT`, datés comme la protection, le statut de détenu et le service ; saisie `SEJOUR` (séjours successifs : `sejourId`, IEP, entrée, sortie, unité, chambre, lit ; `sejourEnCours`) ; accès « Fiches patient » = `PATIENT` et `SEJOUR` ; réglage `fichePatient.services[code]` { psychiatrie (par défaut oui), libellés des médecins } au journal du socle (`optionsFiche`) ; onglet *Patients* : *+ Nouvelle fiche* (patients sans mesure), fiche en lecture seule sans l'accès, séjours, référents parmi les comptes du service (liste de surveillance) ou nom libre ; panneau *Fiche patient par service*.
4. **Import de la liste des patients du DPI (0.6.4).** `import-patients.js` (logique, sous Node) et `ui-import-patients.js` (onglet *Patients → Importer la liste du DPI…*) : lecture CSV (UTF-8 puis Windows-1252) ou Excel, lecteur CSV commun déplacé dans `xlsx.js` ; colonnes proposées par nom d'en-tête, colonnes découpées (« Nom prénom » selon les règles du § 8, « Loc. » en chambre et lit) ; unités du DPI → services (ou écartées), valeurs du sexe ; aperçu (nouveaux, mis à jour, écarts gardés, doublons à confirmer, découpages à vérifier, erreurs, absents) ; saisies `PATIENT` et `SEJOUR` marquées `origine: 'import-dpi'` en un lot annulable ; source de chaque champ (`sources` : import ou saisie) : une saisie dans Tessera n'est jamais écrasée ; découpage validé gardé (`decoupageNom`) ; sortie des absents seulement si cochée. Modèle mémorisé (`config.importPatients.modeles`, établissement ou service ; noms de colonnes et correspondances seulement), groupe d'export des réglages « Module Patients ».
5. **Reprise de l'historique** : imports du registre et du classeur en deux lots ; contrôle (registre, cumuls et échéances identiques avant et après) ; droit d'accès réunissant tous les modules ; durée de conservation d'après la dernière activité dans tous les modules.

8. Imports

Liste des patients exportée du DPI (décisions des 4 au 7 novembre 2026)

Export CSV fait par un agent depuis le DPI, importé dans Tessera (requête planifiée et flux HL7 PAM écartés pour l'instant). Fichier lu dans le navigateur, jamais copié sur le partage.

1. **Lecture** : CSV ou Excel ; UTF-8 puis Windows-1252 (comme `ui-import.js`) ; ligne d'en-tête repérée ; lignes vides ignorées.
2. **Attribution des colonnes** : pour chaque champ, une colonne ou « Non fourni par l'export : à renseigner dans Tessera » (choix automatique pour un champ sans colonne). **Une colonne peut être attribuée à plusieurs champs et découpée.**
3. **Correspondance des valeurs** : sexe, codes d'unité du DPI vers les services, formats de date.
4. **Aperçu** : patients nouveaux, fiches modifiées (ancienne → nouvelle valeur), doublons possibles, lignes de découpage ambiguës, patients absents de la liste.
5. **Enregistrement** en un lot annulable, tracé.

- **Modèle d'import mémorisé** (par établissement, ou par service si les exports diffèrent), dans l'export des réglages.
- **Source de chaque champ** : un champ « à renseigner dans Tessera » n'est jamais touché par l'import ; une correction manuelle d'un champ importé n'est pas écrasée (écart signalé).
- **Patients absents de la liste** : sortie proposée, jamais appliquée d'office.

Export « patients présents » de Sillage (exemple fourni le 7 novembre 2026, colonnes confirmées par le porteur) : séparateur `;`, encodage Windows-1252.

Colonne	Champ	Remarque
`IEP`	Numéro de séjour	Identifiant d'épisode, change à chaque séjour
`Sexe`	Sexe	`F` / `M`
`Nom prénom`	Nom, prénom, nom de naissance	« NOM PRENOM Née NOMNAISSANCE » ; tout en majuscules sauf `Né` / `Née`, mot-clé toujours présent
`Né·e le`	Date de naissance	JJ/MM/AAAA
`Entrée`	Date d'entrée	JJ/MM/AAAA
`RM`, `RH`	Unité médicale, unité d'hébergement	Codes d'unité, à faire correspondre aux services
`Loc.`	Chambre et lit	`HD412-U` = chambre 412, lit U (unique) ; P = côté porte, F = côté fenêtre
`Médecin responsable`	Non importé par défaut	Distinct des médecins de 1re et 2e ligne
`Commentaires`	Non importé	Texte libre, risque de données cliniques

Découpage du nom : (1) ce qui suit `Né` / `Née` est le nom de naissance ; (2) si le texte avant le mot-clé commence par le nom de naissance, ce nom est le nom d'usage et le reste le prénom ; (3) sinon, le dernier mot est le prénom (trait d'union conservé) ; (4) toute ligne découpée par la règle 3 avec plus de deux mots est signalée ; le découpage validé est gardé pour la fiche. Prénom affiché avec initiale majuscule, nom en capitales.

IEP et IPP : sans IPP dans l'export, une réadmission arrive avec un nouvel IEP et doit être rattachée par la détection des doublons (nom, prénom, nom de naissance, date de naissance, sexe), avec confirmation. Le registre exige l'identifiant du patient : ajout de l'IPP à l'export à demander à l'administrateur de Sillage ; à défaut, saisie dans Tessera.

Reprise de l'historique des mesures (décision du 5 novembre 2026)

Mise en production **à partir de zéro donnée**. Les deux imports d'Isocont sont conservés et peuplent à la fois Patients et Isocont :

- import à partir du **registre du service** (CSV ou Excel de n'importe quel outil) et, comme cas particulier, **import du classeur V1.3** ; mécanique inchangée (services absents, compteurs, contrôle de numérotation, aperçu, décisions par mesure) ;
- **deux lots** (Patients : identité, protection, mode et historique ; Isocont : mesures, pauses, JLD, tiers, surveillance) liés par un identifiant d'import commun : annulation conjointe, pas de doublon si relancé ; Isocont transmet ses propositions de fiches à Patients, qui les enregistre lui-même ;
- rattachement par l'IPP, puis par la détection des doublons ;
- **ordre conseillé** : historique des mesures, puis liste des patients présents (qui complète séjour et chambre) ;
- contrôle : cumuls, échéances et registre recalculés, comparés au document importé.

9. Stockage sur le partage

```
TesseraSante\
  coffre.json                  clés publiques, enveloppes, comptes chiffrés, révision
  coffre-historique\           30 dernières versions du coffre
  socle\evenements\AAAA\MM\    saisies du socle
  reperes\  preferences\  quarantaine\
  modules\patients\evenements\AAAA\MM\
  modules\isocont\evenements\AAAA\MM\   (+ instantanes\, formation\)
  modules\agenda\evenements\AAAA\MM\
  echanges\                    seulement si la publication est activée
  exports\<service>\<catégorie>\
```

- Sauvegardes, dossier de secours, copie sous scellé et réparation travaillent sur l'ensemble du dossier.
- Contrôle d'intégrité journal par journal : socle, puis chaque module. Chaque journal a ses repères, ses instantanés et sa quarantaine ; les prises d'acte (saisies `INTEGRITE`, journal du socle) désignent le journal de chaque repère.
- **Aiguillage des saisies** (0.3.1) : journal du socle pour les traces des comptes et des clés (`COMPTE`), les prises d'acte d'intégrité et les réglages communs (`etablissement`, `services`, `roles`, `modulesServices`) ; journal de la saisie visée pour une annulation ; journal du module pour tout le reste, imports compris. Chaque journal a son propre chaînage.
- **État d'un module** = réduction du journal du socle et du journal du module ensemble (réglages communs + saisies du module).
- **Jeu de formation** : `modules\<module>\formation\`, un seul journal (fictif), comme avant.
- **Copie de sauvegarde au format d'Isocont** (`evenements\` à la racine) : restaurée dans `modules\isocont\` (reprise d'une sauvegarde d'Isocont).

10. Hébergement

- **Phase pilote** : version publiée par le porteur sur GitHub Pages (`https://rodtrip2.github.io/tessera-sante/`) ; seul le code y est hébergé ; le porteur maîtrise les évolutions jusqu'à maturité des modules ; **double authentification activée sur le compte GitHub**, branche principale protégée. Usage avec des données réelles soumis à l'accord du

service informatique, du responsable de la sécurité informatique et du délégué à la protection des données.
- **À maturité** : copie d'une version étiquetée sur l'intranet HTTPS de l'établissement ; clés de sécurité à réenregistrer (liées au nom de domaine) ; données inchangées.

11. Plan de développement

Sept étapes, chacune publiable seule avec tous les tests au vert.

1. **Découper sans changer le comportement.** `model.js` → `noyau-journal.js` (réduction, annulations, chaînage, conflits) + `droits.js` (rôles, accès, services) + modèle Isocont. `session.js` → session du socle + appels au module. `ui.js` → coquille + vues Isocont. Les tests existants passent sans modification.
2. **Contrat de module et registre des modules** (`SIC.modules` ou équivalent). Isocont se déclare ; `ui-roles.js` , les réglages et la formation lisent ce que le module déclare. Interface identique.
3. **Comptes et services au socle.** Réglages `services` et `roles` au journal du socle ; activation des modules par service.
4. **Trousseau.** Clés personnelles, clé de plateforme, clés de module par générations ; code de secours unique.
5. **Module Patients.** Fiches, modes, séjours, fusion et doublons sortent d'Isocont ; import de la liste du DPI ; imports du registre et du classeur adaptés pour peupler les deux modules ; registre, cumuls et échéances identiques avant / après.
6. **Module Agenda.** Frontière, activation par service, premier échange (échéances JLD en lecture à la volée), écran d'accueil des modules.
7. **Kit de déploiement.** Fiche technique, éléments RGPD (un traitement par module), guides d'installation et d'utilisation.

État du plan (au 7 novembre 2026) :

Étape	État
Reprise du code d'Isocont 1.67	Faite : historique d'Isocont relié à celui de Tessera (commit de fusion), 139 tests repris au vert
1. Découper	**Faite (0.2.0).** `model.js` → `noyau-journal.js` (tri, lots, annulations et rétablissements, chaînage, conflits, intégrité), `droits.js` (rôles, accès, services, visibilité du journal et des onglets), `exports-rangement.js` (rangement et purge des exports) + modèle d'Isocont, même interface. `session.js` → session du socle, seul point d'appel au module : `moduleActif()` (reconstitution de l'état, réglages par défaut). `ui.js` → coquille + `ui-isocont.js` : vues et fenêtres, libellés du journal (`SIC.ui.journal`), panneaux d'administration (`SIC.ui.admin`). Correction en chemin : ordre des saisies d'une même milliseconde (0.1.1). Garde-fou : `tests/interface-noms.test.js`

Étape	État
2. Contrat de module et registre	**Faite (0.3.0).** `modules.js` : registre (déclaration contrôlée du contrat, interface ajoutée par le fichier d'interface du module, besoins non satisfaits, migrations à faire). `module-isocont.js` : contrat d'Isocont (identité, accès, droits, saisies, réglages, onglets, migrations — aucune —, formation, fourniture `isocont.echeancesJLD` v1 décrite, besoins — aucun avant le module Patients). `droits.js` devient un moule (`creer`) : règles au socle, accès et rôles fournis par le module. Session, coquille (onglets, vues, journal, administration), *Rôles et accès*, export des réglages et mode formation lisent le contrat. Interface identique (comparée dans Chrome à Isocont 1.67)
3. Comptes et services au socle	**Faite (0.4.0).** Journal du socle et rangement par journal (0.3.1) : `socle\` , `modules\isocont\` (jeu de formation dessous) ; réglages communs (établissement, services, rôles) au journal du socle ; intégrité, copies, réparation, quarantaine, dossier de secours et déplacement journal par journal. Activation des modules par service (0.4.0) : réglage `modulesServices` au journal du socle, services visibles et saisies refusées selon l'activation (`droits.js` : `activation` , `serviceActif` , `consulteInactifs` ; contrat : `saisies.cibles`), panneau *Modules activés par service*
4. Trousseau	Fait. Briques (0.4.1), coffre version 2 (0.4.2), migration et retour à la clé unique (0.4.3), remise et retrait des clés selon les droits (0.4.4) : clé du module selon le rôle et l'activation, nouvelle génération automatique au retrait, nouvelle clé maîtresse au retrait d'un administrateur, clé personnelle créée par l'agent, postes ouverts mis à jour, réémission des clés. Corrections de la relecture indépendante (0.4.5). Attestation du coffre, renouvellement après restauration, compromission, panneau *Intégrité du coffre* (0.5.0)
5. Module Patients	En cours. Journal et clé propres (0.6.0) ; fiche reconstituée par Patients, lue à la volée par Isocont, lots liés (0.6.1) ; droits de la fiche et onglet en tête (0.6.2) ; fiche commune complète et séjours (0.6.3) ; import de la liste des patients du DPI (0.6.4). Reste : reprise de l'historique (sous-étape 5, § 7)
6 et 7	À faire

Restent propres à Isocont dans la coquille : quelques aides d'affichage liées aux patients et aux mesures (`patient` , `lieu` , réglages d'affichage de l'accueil), à reprendre avec le module Patients (étape 5) ; textes (titre, « Informations techniques », fichier lisez-moi du jeu de formation), à généraliser avec l'écran d'accueil des modules (étape 6). À l'étape 3 : le panneau « Règles légales et sécurité » d'Isocont porte aussi le verrouillage après inactivité, réglage du socle.

Points de vigilance : `ui.js` est le découpage le plus long, à faire par blocs d'écran ; la mise en place du trousseau (étape 4) doit être testée sur une copie complète avant tout dossier réel.

12. Décisions du porteur

Question	Décision	Date
Nom et dépôt	Tessera Santé, dépôt `rodtrip2/tessera-sante` (aucun résultat à l'INPI)	4–5 nov. 2026
Retrait d'un droit	Nouvelle génération de clé ; les autres agents et les nouveaux gardent tout l'historique	4 nov. 2026
Rôles et réglages	Administration unique : onglet général + un onglet par module	4 nov. 2026

Question	Décision	Date
Données d'Isocont	Déplacées sous `modules\isocont\` (outil non déployé)	4 nov. 2026
Code de secours	Un seul pour la plateforme	4 nov. 2026
Premier module nouveau	L'Agenda, après Patients	4 nov. 2026
Connexion Windows	Au niveau du socle, plus tard (voir `evolutions-futures.md`)	4 nov. 2026
Cloisonnement entre services	Par l'interface	4 nov. 2026
Rôle selon le service	Pas dès le départ, mais prévu dans le modèle	4 nov. 2026
Échanges entre modules	Lecture à la volée ; publication activable plus tard	4 nov. 2026
Identité dans l'agenda	Au choix : aucune, initiales, IPP, nom complet	4 nov. 2026
Création des fiches patient	Paramétrable ; IDE, secrétaire, cadre par défaut	4 nov. 2026
Patients sans mesure	Saisie manuelle et import CSV exporté du DPI	4 nov. 2026
Champs de la fiche	Voir § 7	5 nov. 2026
Colonnes de l'import	Attribuées à l'import ; champ non fourni → saisi dans Tessera ; colonne découpable	5–6 nov. 2026
Reprise de l'historique	Départ à zéro, imports du registre et du classeur peuplant Patients et Isocont	5 nov. 2026
Hébergement du pilote	GitHub Pages, double authentification	5 nov. 2026
Export de Sillage	Colonnes et règles du § 8	7 nov. 2026
Activation d'un module par service	Sans réglage : tous les services ; consultation d'un service désactivé par le cadre et l'administrateur ; service créé après une désactivation : à activer (proposition retenue en 0.4.0, à confirmer)	7 nov. 2026
Lien avec le dépôt d'Isocont	Code repris de la version 1.67 avec son historique (commit de fusion) ; numérotation propre à Tessera à partir de 0.1.0	7 nov. 2026
Menace couverte par le trousseau	Un agent qui garde un accès en écriture au partage fait partie de la menace : coffre attesté ; renouvellement de toutes les clés après une restauration ; « Tout renouveler » en cas de compromission	7 nov. 2026

Question	Décision	Date
Fiches patient et AS	L'AS ne crée ni ne corrige les fiches patient (IDE, secrétaire et cadre par défaut, réglable) ; elle garde le mode d'hospitalisation saisi avec la mesure	7 nov. 2026
Onglet Patients	Onglet à part, en tête de la barre, en attendant l'écran d'accueil des modules	7 nov. 2026
Séjours	Historique des séjours successifs (IEP, entrée, sortie, unité, chambre et lit) ; la fiche montre le séjour en cours	7 nov. 2026
Discipline d'un service	Réglage par service « service de psychiatrie » (coché par défaut pour les services existants), libellés modifiables	7 nov. 2026

Questions restantes : ajout de l'IPP à l'export de Sillage (à demander à l'administrateur du DPI) ; contenu de l'Agenda au-delà des échéances JLD (à définir avec les équipes) ; devenir du dépôt `Suivi-mesures-isocont` une fois Tessera en service.